# Automated Vulnerability Discovery & Remediation Pipeline

*WordPress + WPScan + GitHub Actions*

Deian Ageu

May 2026

## Overview

This report documents the lab I built for the DevSecOps module. The brief was to stand up a containerised WordPress deployment, scan it with WPScan, fix what the scanner found through patching and hardening, rebuild the image, and verify the fixes with an automated re-scan — all of this wired into GitHub Actions so a future change cannot regress the security posture without being noticed.

The end result is a small pipeline that starts from an intentionally outdated WordPress 5.7.0 image with 46 known core CVEs plus a handful of misconfigurations, and ends with a hardened image (WordPress 6.8.3, restricted Apache + PHP, removed default plugins) where WPScan reports essentially nothing. The hardened image is published to both Docker Hub and GitHub Container Registry, and a workflow re-scan is automatically triggered after every successful build so the "before / after" comparison is always reproducible.

Repo: https://github.com/DeianGG/DevSecOps Image: https://hub.docker.com/r/deiangg/devsecops

| | Baseline (`wordpress:5.7.0`) | Hardened (`deiangg/devsecops:hardened`) |
|---|---|---|
| WordPress core CVEs | 46 | 0 |
| Vulnerable plugins | 1 (Akismet, generic match) | 0 (plugin removed) |
| User enumeration | `admin` exposed | "No Users Found" |
| XML-RPC endpoint | open | denied (HTTP 403) |
| `readme.html` / `license.txt` | reachable | removed / blocked |
| Server / PHP headers | `Apache/2.4.38 (Debian)`, `PHP/7.4.16` | `Apache`, no `X-Powered-By` |
| Security response headers | none | 5 (CSP-adjacent set) |

# 1. Environment Setup

## 1.1 Steps taken

I started by sketching the directory layout from the brief (`docker/`, `src/`, `scans/`, `.github/workflows/`) and dropping a `.gitignore` in before the first commit so that the `docker/.env` file (the only place credentials live) could never be accidentally pushed.

The vulnerable baseline lives in `docker/docker-compose.yml`. I deliberately pinned `wordpress:5.7.0-php7.4-apache` — released in March 2021, so by now it predates fourteen WordPress security releases — alongside `mariadb:10.6`. Both services are on a dedicated bridge network, the database port is *not* published to the host, and the WordPress credentials are read from `docker/.env` (template in `.env.example`). WordPress is exposed on port 8090 to leave 8080 free, since I had another service running there.

Bringing the stack up is the usual `docker compose up -d`. To keep the rest of the flow scriptable I avoided the browser installer and instead `POST`ed straight to `/wp-admin/install.php?step=2`. That same trick is what the GitHub Actions workflow later reuses, which means CI and local runs converge to the same site state.

For the manual baseline scan I used the official `wpscanteam/wpscan` Docker image rather than installing Ruby + gems locally — fewer moving parts, same scanner. I asked it for everything I cared about: user enumeration, vulnerable plugins, vulnerable themes, exposed config backups, and exposed DB dumps. The full output went into `scans/before.txt`.

## 1.2 Challenges encountered

A few things did not work the first time. None were dramatic but they cost real time and were the bits I actually learned from.

The first one was straightforward: `mysql:5.7` has no `linux/arm64/v8` manifest, and I'm on an Apple Silicon machine. I swapped MySQL for `mariadb:10.6`, which is wire-compatible with WordPress's MySQL client and has a native ARM64 build. The old `wordpress:5.7.0-php7.4-apache` image also lacks an ARM64 manifest, so for the vulnerable baseline I forced `platform: linux/amd64` and let Docker Desktop emulate it under Rosetta. This is acceptable because the baseline is something I *want* to be old and "wrong"; the hardened image is built fresh as multi-arch.

The second one took me a while to figure out. I added `ServerTokens Prod`, `ServerSignature Off`, and `Header always unset Server` to my hardening config and the `Server: Apache/2.4.62 (Debian)` header stubbornly refused to go away. After staring at `apache2ctl -M` and reading what was actually loaded from `conf-enabled/`, I realised the problem was alphabetical order: Debian's stock `security.conf` was loading *after* my `security-extra.conf`, so it was overwriting my settings rather than the other way around. I renamed mine to `zz-security-hardening.conf` so it sorts last and wins. After that the header was just `Server: Apache`.

The third one was similar in spirit. I wrapped a `RewriteRule` to block `?author=<id>` enumeration but it kept letting requests through. The fix was a context issue: `RewriteRule` at server scope doesn't apply to URL-encoded query strings the way I expected. Once I wrapped it in `<Directory /var/www/html>` it worked.

The last one was more of a "WPScan quirk" than a real bug. Even after I blocked `/wp-content/plugins/akismet/` behind a 403, WPScan kept reporting Akismet as vulnerable. It turned out the scanner falls back to a generic "any version" match when it cannot read the plugin's `readme.txt`, and then optimistically lists the worst CVE for the family. Rather than fight the scanner, I removed Akismet entirely from the image — WordPress runs fine without it and the report becomes a clean "No plugins Found".

### 1.3 Final stack

| Component | Baseline | Hardened |
|---|---|---|
| WordPress | `wordpress:5.7.0-php7.4-apache` | `wordpress:6.8.3-php8.3-apache` |
| Database | `mariadb:10.6` | `mariadb:10.6` |
| Apache | 2.4.38 (Debian) | 2.4.62, `ServerTokens Prod` |
| PHP | 7.4.16 | 8.3, `expose_php=Off`, dangerous functions disabled |
| Exposed port | `8090 → 80` (container as root) | `8091 → 8080` (container fully as `www-data`) |

## 2. Findings Overview

### 2.1 Key vulnerabilities

WPScan returned 48 distinct findings against the baseline. They group naturally into three buckets — core CVEs, plugin CVEs, and misconfigurations — and it's worth looking at each separately because they call for different mitigations.

The **WordPress core** bucket is by far the largest: 46 CVEs, all of which are explained by a single fact, namely that the image is pinned at 5.7.0. That release came out in March 2021 and every release since (5.7.1, 5.7.2, …, 6.8.3) has patched something. The CVEs that matter most for this lab, in roughly descending impact, are:

- **CVE-2022-21661** and **CVE-2022-21664** — two SQL injection sinks reachable through `WP_Query` and `WP_Meta_Query`. Both are unauthenticated if a plugin or theme passes user input into either method, which is extremely common.

- **CVE-2022-21663** — object injection on multisite super-admin actions. The deserialisation path here is the classic precursor to PHP RCE through a gadget chain.
- **CVE-2021-29447** — XXE in the Media Library when PHP 8 parses the upload, allowing an authenticated user to read arbitrary files or pivot the host into SSRF.
- **CVE-2020-36326** — PHPMailer object injection. The mail-sending path can be coerced into deserialising attacker-controlled data.
- **CVE-2022-3590** — unauthenticated blind SSRF via DNS rebinding, which is the kind of bug that lets you reach internal cloud metadata endpoints.

The full list lives in `scans/before.txt`; I haven't reproduced every one of them here because the pattern is the same: missing security release.

The **plugin** bucket is just Akismet. WPScan flagged it as vulnerable but the title says "Akismet 2.5.0–3.1.4", and the bundled Akismet is much newer. This is the false-positive case I described in §1.2 — when WPScan can't read `readme.txt`, it falls back to a generic match. Removing the plugin entirely was simpler than trying to make WPScan see the version through the 403 block.

The **misconfiguration** bucket is where most of the interesting attack surface lives, because misconfigurations don't get patched the way CVEs do — they stay around until someone explicitly removes them:

1. `?author=<id>` user enumeration. WPScan discovered the `admin` username by walking author IDs and watching for redirects to `/author/<name>/`. It then confirmed it through the login error oracle (`/wp-login.php` returns subtly different messages for unknown vs. known usernames).
2. `xmlrpc.php` enabled. Beyond letting an attacker call legacy XML-RPC methods, this endpoint accepts `system.multicall` payloads which wrap arbitrary numbers of `wp.getUsersBlogs` calls in one HTTP request — a credential-stuffing amplifier that defeats naive lockout policies. The `pingback.ping` method is also a known SSRF reflector.
3. `readme.html` reachable. Tells the attacker the exact WordPress version on the first request.
4. `Server` and `X-Powered-By` headers. Confirm Apache 2.4.38 and PHP 7.4.16, both of which have their own published CVEs.
5. External `wp-cron.php`. Any unauthenticated request can trigger scheduled jobs, which is a known DoS amplifier when those jobs are expensive.

## 2.2 Risk assessment

Translating CVSS-style scoring into a small lab is always a bit hand-wavy, but the rough picture is this:

| Risk | Likelihood | Impact | Severity |
|---|---|---|---|
| Unauthenticated SQLi via `WP_Query` / `WP_Meta_Query` → DB takeover | Medium (depends on plugin sink) | High | **Critical** |
| Object Injection → PHP RCE | Medium | High | **Critical** |
| Credential brute-force amplified by `xmlrpc.php` + leaked `admin` | High | High | **High** |
| Authenticated XXE / file disclosure | Medium | Medium | **High** |
| Stored XSS chains → admin-session hijack | High | Medium | **High** |
| SSRF via pingback / DNS-rebinding | Medium | Medium | **Medium** |
| Information disclosure (versions, email oracles) | High | Low | **Low** |

What I tried to capture in this table is that "lots of CVEs" doesn't translate one-to-one into "lots of severe risk" — most of the SQLi bugs need a vulnerable plugin sink that we don't ship with, and a lot of the XSS bugs need an authenticated contributor. But the *combination* of user enumeration + `xmlrpc.php` + the admin oracle in `wp-login.php` is a realistic unauthenticated path to compromise, and that one I take seriously.

## 2.3 How attackers could exploit them

To make this concrete, I worked through three plausible attack paths an attacker might walk down with this baseline. They aren't meant to be the only possibilities — they're meant to show how a few independent findings can compose into a real-world chain.

**Path A — full takeover from the public homepage.** The attacker requests `/?author=1`, which redirects to `/author/admin/`, leaking the username. They then script a brute-force against `xmlrpc.php`, using `system.multicall` to wrap a few thousand `wp.getUsersBlogs` attempts per HTTP request. Because WordPress doesn't rate-limit XML-RPC the way it (loosely) rate-limits the login form, this evades any "lockout after N failures" detection. Once a password works, they log into `/wp-admin`. From there they trigger the Media Library XXE (CVE-2021-29447) to read `wp-config.php`, recovering both the DB password and the `AUTH_KEY` constants. Game over.

**Path B — unauthenticated DB exfiltration through a vulnerable plugin.** The attacker fingerprints WordPress 5.7.0 from `/readme.html` and the meta-generator tag. They scan for any installed plugin

that passes user input into `WP_Query`'s `meta_query` (this is common enough that lists exist), then drive CVE-2022-21664 to blind-exfil rows out of `wp_users.user_pass`. No login required.

**Path C — SSRF pivot.** The attacker calls `xmlrpc.php` with `pingback.ping`, pointing at an internal target (e.g. `http://169.254.169.254/latest/meta-data/` on a cloud host). WordPress dutifully proxies the HTTP request. Timing differences reveal which internal services exist; with CVE-2022-3590 the attacker can rebind DNS mid-request to bypass URL allow-lists.

---

# 3. Remediation Steps

## 3.1 Patching and updating

The single highest-leverage change was rebasing the image onto a patched WordPress release: 5.7.0 → 6.8.3. WordPress core patches are cumulative, so moving to the current branch closed all 46 baseline CVEs in one rebuild — every SQLi, every XXE, every Object Injection, every stored-XSS finding. I bumped PHP at the same time (7.4 → 8.3), which removed the disclosed runtime version and pulled in upstream PHP security fixes. MariaDB 10.6 was already a maintained release so I didn't touch it.

For plugins, the only one shipped by default was Akismet, and I deleted it from the image rather than upgrading it — both because the false-positive in WPScan was annoying and because the lab doesn't need it. For themes, the default install ships several years-worth of "twenty-something" themes; I kept only `twentytwentyfour` because WordPress needs at least one theme present to serve a page.

## 3.2 Hardening measures

Patching closes known CVEs. Hardening reduces the *surface* the next round of CVEs has to land on. I tried to apply layers at four levels — Apache, PHP, WordPress constants, and the image itself — so that a bypass at one layer is caught at the next.

At the Apache layer (`docker/hardening/security.conf`) I:

- Set `ServerTokens Prod`, `ServerSignature Off`, `TraceEnable Off`, `FileETag None`, and added `Header always unset Server` + `Header unset X-Powered-By` so the response no longer advertises the tech stack.
- Added a small set of security headers — `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: geolocation=(), microphone=(), camera=()`, and `X-XSS-Protection`. These are not glamorous but they take a chunk off the post-XSS exploitation tree.
- Denied direct access to `xmlrpc.php` (closes the brute-force amplifier and the SSRF reflector), `wp-config.php` (defence in depth in case some misconfigured fronted serves it as text), all

dotfiles (so an accidentally-deployed `.git/` or `.env` is not reachable), and any `readme*` / `license*` / `changelog*` document.

- Wrapped a rewrite rule in `<Directory /var/www/html>` that returns 403 for any request whose query string contains `author=<digits>`, so the author-enumeration trick from §2 stops working.
- Inside the `wp-content/uploads/` directory I denied execution of `.php`, `.phtml`, and `.phar`, which is the standard mitigation against web-shell upload chains.

At the PHP layer (`docker/hardening/security.ini`) I disabled `expose_php`, turned off `allow_url_fopen` and `allow_url_include` (closes a class of RFI bugs), set strict session cookies (`httponly`, `samesite=Strict`, `use_strict_mode`), and disabled a list of dangerous functions (`exec`, `shell_exec`, `system`, `proc_open`, `popen`, `passthru`, `curl_multi_exec`, `parse_ini_file`, `show_source`).

At the WordPress layer I injected hardening constants through the `WORDPRESS_CONFIG_EXTRA` environment variable so they get appended to `wp-config.php` at runtime — `DISALLOW_FILE_EDIT` (closes the admin-UI code editor, which is otherwise a free RCE once an admin session is compromised), `DISALLOW_FILE_MODS` (same idea for plugin/theme installs), `WP_DEBUG=false`, `WP_POST_REVISIONS=5`, `EMPTY_TRASH_DAYS=7`. The full set is documented in `docker/wp-config-placeholder.txt`.

At the image layer I removed the OS tools an attacker would otherwise find useful for living off the land (`wget`, `less`, `nano`, `vim-tiny`), set all WordPress files to mode `0644` and directories to `0755` owned by `www-data`, and added a `HEALTHCHECK` against `/wp-login.php` so that any orchestrator running the image gets a real liveness signal.

The last thing I did at this layer was take the container fully non-root. The official `wordpress` image starts Apache as UID 0 because it has to bind port 80, and only then drops the worker processes to `www-data`. The simplest way around this is to teach Apache to listen on an unprivileged port instead: a couple of `sed` lines patch `/etc/apache2/ports.conf` from `Listen 80` to `Listen 8080` and the matching `VirtualHost`, after which the `apache2` master can run as `www-data` from the start. I also `chown`ed `/var/run/apache2`, `/var/log/apache2` and `/var/lock` so the runtime directories are writable by the new UID, and then added a single `USER www-data` line at the end of the Dockerfile. The host-side port mapping in `docker-compose.hardened.yml` becomes `8091:8080` so the user-facing URL is unchanged. After the rebuild, `docker exec wp-site ps -eo user,comm` shows every process — including the Apache master — as `www-data`. No CAP_NET_BIND_SERVICE trickery needed.

## 3.3 Before / after scan evidence

Both reports are versioned in `scans/`. The relevant parts of each are below.

**Baseline (`scans/before.txt`, abbreviated):**

```
[+] Headers
 |  - Server: Apache/2.4.38 (Debian)
 |  - X-Powered-By: PHP/7.4.16

[+] XML-RPC seems to be enabled: http://.../xmlrpc.php
[+] WordPress readme found: http://.../readme.html
[+] WordPress version 5.7 identified (Insecure, released on 2021-03-09).
 | [!] 46 vulnerabilities identified:
 | [!] Title: WordPress 5.6-5.7 - Authenticated XXE Within the Media Library
 | [!] Title: WordPress < 5.8.3 - SQL Injection via WP_Query
 | [!] Title: WordPress 4.1-5.8.2 - SQL Injection via WP_Meta_Query
 | [!] Title: WordPress < 5.8.3 - Super Admin Object Injection in Multisites
 | [!] Title: WordPress 3.7 to 5.7.1 - Object Injection in PHPMailer
 | ... (41 more)

[+] akismet
 | [!] 1 vulnerability identified (Akismet 2.5.0-3.1.4 - Stored XSS)

[+] admin
 | Found By: Author Id Brute Forcing (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)
```

**Hardened (`scans/after.txt`, full body):**

```
[+] Headers
 |  - Server: Apache
 |  - Referrer-Policy: strict-origin-when-cross-origin
 |  - Permissions-Policy: geolocation=(), microphone=(), camera=()

[+] The external WP-Cron seems to be enabled (60% confidence)

[+] WordPress version 6.8.3 identified.
 |   (no vulnerability block)

[i] The main theme could not be detected.
[i] No plugins Found.
[i] No themes Found.
[i] No Config Backups Found.
[i] No DB Exports Found.
[i] No Users Found.
```

**Delta:**

| Metric | Before | After |
|---|---|---|
| WP core vulnerabilities | 46 | 0 |
| Plugin vulnerabilities | 1 (false positive) | 0 |
| Identified users | `admin` | none |
| `xmlrpc.php` reachable | yes | no (HTTP 403) |
| `readme.html` reachable | yes | no (HTTP 403) |
| `Server` header leaks version | yes | no |
| `X-Powered-By` header present | yes | no |
| Security response headers | 0 | 5 |

The only thing the post-hardening scan still flags is the external `wp-cron.php`, at 60% confidence. I left it open in this lab so the site can run with one command; the production fix is `define('DISABLE_WP_CRON', true)` plus a real system cron driver — I mention this in the README as a known limitation.

---

# 4. Fixed Image Build

## 4.1 GitHub repository

- Repo: https://github.com/DeianGG/DevSecOps
- Workflow runs: https://github.com/DeianGG/DevSecOps/actions
- Baseline scan: `scans/before.txt`
- Hardened scan: `scans/after.txt`

## 4.2 Published images

I pushed to both Docker Hub and GitHub Container Registry — partly because the brief asks for both, and partly because mirroring removes single-vendor risk if one registry has an outage during a deploy.

| Registry | Reference |
|---|---|
| Docker Hub | https://hub.docker.com/r/deiangg/devsecops |
| GHCR | `ghcr.io/deiangg/devsecops` |

The published tags are:

- `:vulnerable` — the upstream `wordpress:5.7.0-php7.4-apache`, re-tagged into my namespace so reviewers can pull the exact baseline I scanned (amd64 only, since the upstream has no ARM64 manifest).
- `:hardened` — the rebuilt and hardened image, multi-arch (linux/amd64, linux/arm64).
- `:latest` — alias of `:hardened`.
- `:sha-<commit>` — immutable per-commit tag, so a deployment can pin a digest instead of trusting a moving tag.

Pull and run:

```
docker run -d --name wp-hardened -p 8091:80 \
  -e WORDPRESS_DB_HOST=<host> \
  -e WORDPRESS_DB_USER=<user> \
  -e WORDPRESS_DB_PASSWORD=<pass> \
  -e WORDPRESS_DB_NAME=wordpress \
  deiangg/devsecops:hardened
```

The image carries OCI metadata labels (`org.opencontainers.image.title`, `…description`, `…source`, `…licenses`) and was built by `docker/build-push-action@v6` with the GitHub Actions cache backend. Build provenance is therefore traceable back to the exact commit on `main`.

---

# 5. Tooling Justification

For each of the tools the brief lists, this is why I think it was the right pick rather than the obvious alternative.

**Docker + Docker Compose.** A two-service WordPress + DB stack does not justify Kubernetes or even Swarm. Compose gives a single declarative file that runs the same on my laptop, in GitHub Actions, and on whatever Linux host I'd deploy to. Anything heavier would have added orchestration complexity without changing the security exercise.

**MariaDB instead of MySQL.** The brief's example workflow uses `mysql:5.7`, but that image has no ARM64 manifest and most newer Mac dev machines are Apple Silicon. MariaDB 10.6 is binary- and wire-compatible with the WordPress MySQL client, has a native ARM64 build, and is actively maintained — a strictly better choice today.

**WPScan.** WordPress-specific scanners outperform generic ones (Nikto, ZAP, Trivy-on-the-image) for this target because WordPress has very particular fingerprints: author-ID enumeration, plugin/theme path probing, the WPScan-curated CVE database. Running it inside the official `wpscanteam/wpscan` container avoided the Ruby + gem install dance the brief's example workflow shows. The WPScan API token (free tier, 25 req/day) is what enriches each finding with the `Fixed in: X` annotation that drives the patch decision — without it the scanner still runs but you're left guessing which release closed which CVE.

**GitHub Actions.** Co-located with the repo, free for public projects, native secret management, and — crucially for this pipeline — able to dispatch other workflows via `actions/github-script`. That

last detail is what lets `build-and-push.yml` automatically trigger `scan.yml` against the freshly pushed image, closing the loop without external glue.

**GitHub Container Registry.** Inherits visibility from the repo (so a public repo gives a public image with no extra config), authenticates with the workflow's auto-provisioned `GITHUB_TOKEN` rather than another secret. Mostly a free convenience.

**Docker Hub.** Required by the brief and still the registry most people look at first. Publishing to both gives reviewers two pull paths.

`docker/build-push-action` **with Buildx + QEMU.** A single step builds for `linux/amd64` and `linux/arm64` from the same Dockerfile, with the GitHub Actions cache backend so subsequent builds aren't paying the full cost.

---

# 6. DevSecOps Strategy

The brief asks how the workflow demonstrates shift-left security. I think of "shift-left" not as a slogan but as a constraint on where in the lifecycle security feedback happens. In this lab the constraint is satisfied because the pipeline puts every security signal *before* the merge, not after the deploy.

In concrete terms:

1. **Scan-as-CI.** Every push to `main` triggers a scan against a clean, ephemeral WordPress instance. The output is uploaded as a build artifact and committed back into `scans/`, so the security posture is part of the same Git history as the code. Asking "when did this CVE first appear?" becomes a `git blame scans/before.txt`.
2. **Closed-loop remediation.** `build-and-push.yml` doesn't just publish a hardened image — its last step uses `actions/github-script` to dispatch `scan.yml` against the image it just pushed, with `scan_label=after`. The build → re-scan cycle has no manual handoff. Whoever next changes the Dockerfile gets a fresh `after.txt` in CI without thinking about it.
3. **Reproducible baseline.** Both images are pinned to exact tags (`5.7.0-php7.4-apache`, `6.8.3-php8.3-apache`) rather than `latest`, so the before/after delta remains meaningful across reruns. The vulnerable image is also published under my namespace (`:vulnerable`), so a reviewer can reproduce the baseline scan without trusting my reported findings.
4. **Defence in depth.** Hardening is applied at four independent layers (base image, Apache, PHP, WordPress constants). A bypass of any one is contained by the next. This isn't accidental — it's the same logic as not relying on a WAF when you could fix the bug.
5. **Least-privilege secret handling.** `WPSCAN_API_TOKEN` and `DOCKERHUB_TOKEN` live as GitHub repository secrets, are masked in workflow logs, and the Docker Hub token is scoped to a single repository. There is no path by which they end up in the image, in the source, or in a public log line.
6. **Provenance.** Each image carries OCI labels that point back at the source repo, is built multi-arch from the same Dockerfile, and is mirrored to two registries.

## 6.1 What I would add if this were a production system

This is a lab, so a couple of things I'd want in production are deliberately out of scope here. If I were extending it:

- A *separate* image-level vulnerability scan (Trivy or Grype) alongside the application-level WPScan, so OS-package CVEs are visible too.
- An SBOM attached to each release (`docker buildx imagetools inspect --format ...`).
- `cosign` signatures + Sigstore for image provenance.
- Move from the floating `:hardened` tag to immutable digest pins in any deploy manifest.
- Fail-the-build thresholds (`wpscan --severity high --threshold 0`) so a high-severity finding blocks the merge rather than being purely informational.

## 6.2 Honest reflection

The part of this lab that surprised me most was how much of the remediation came from a single decision — bumping the base image version. Forty-six core CVEs disappeared without any clever code change. The hardening work I did on top of that closed maybe a dozen extra findings that you'd never get from a version bump alone (misconfigurations, default plugins, exposed paths). The lesson I'm taking away is that *patching* and *hardening* are different problems, both worth doing, but the relative ROI depends on how stale the baseline is. On a recent baseline, version bumps don't move the needle and hardening dominates; on a stale baseline like this one, the bump is overwhelmingly the highest-leverage action.

The other thing I noticed is that the automated re-scan loop is the part of the pipeline that has the most "DevSecOps" character, but it's also the smallest amount of code — a single `actions/github-script` step. The hard work is in the Dockerfile and the Apache config; the automation that proves it stays fixed is almost free once the rest is in place. That ratio feels right for any future DevSecOps work I do.

---

# Appendix A — Repository layout

```
DevSecOps/
├── README.md
├── docker/
│   ├── docker-compose.yml              # vulnerable baseline
│   ├── docker-compose.hardened.yml     # hardened deployment
│   ├── Dockerfile                      # hardened image definition
│   ├── wp-config-placeholder.txt       # documented wp-config hardening
constants
│   ├── hardening/
│   │   ├── security.ini                # PHP overrides
│   │   └── security.conf               # Apache hardening
│   └── .env.example
├── scans/
│   ├── before.txt
│   └── after.txt
├── src/notes.txt
└── .github/workflows/
    ├── scan.yml
    └── build-and-push.yml
```

# Appendix B — Required GitHub secrets

| Secret | Source | Used by |
|---|---|---|
| `WPSCAN_API_TOKEN` | https://wpscan.com → Profile → API Token | `scan.yml` |
| `DOCKERHUB_TOKEN` | https://hub.docker.com → Account Settings → Security → New Access Token (RWD scope) | `build-and-push.yml` |

`GITHUB_TOKEN` is provided by Actions automatically and used to push to GHCR.